Open in Colab

Reading & Writing Text Files

Writing A String to A Text File

To write some text to a file call builtin `open` function with two parameters:

- a file name
- **"w"**, indicating that the file is opening in **write** mode.

This function call will create a file on your file system. And return a Python file object.

In [1]:
```
open("myfile.txt", mode="w")
```

Out[1]:
```
<_io.TextIOWrapper name='myfile.txt' mode='w' encoding='UTF-8'>
```

Then use `write` method to write a string to that.

In [2]:
```
open("myfile.txt", mode="w").write("Hello world!")
```

Out[2]:
```
12
```

Now a text file named `myfile.txt` with content `"hello world!"` is created.

Warning: If the file already exist then its content is overwritten. If you don't want to overwrite the file use `mode="x"`.

Reading A Text File As A String

To read the contents of a text file call the `open` function. And call its `read` method.

In [3]:
```
open("myfile.txt", mode="r").read()
```

Out[3]:
```
'Hello world!'
```

If the file does not exist then `FileNotFoundError` is raised.

Here, `mode="r"` is default parameter. So you can ommit `mode="r"` when reading text files.

In [4]:
```
open("myfile.txt").read()
```

Out[4]:
```
'Hello world!'
```

Appending A String to A File

Openning a file using `mode="w"` will erase the content of the file if it is already exist. If you want to append a text and keep the previous content, use `mode="a"` instead.

In [5]:
```
open("myfile.txt", "a").write("\nHow are you?")
```

Out[5]:
```
13
```

In [6]:
```
open("myfile.txt", mode="r").read()
```

Out[6]:
```
'Hello world!\nHow are you?'
```

If you want to read only a number of bites than:

In [7]:
```
open("myfile.txt", mode="r").read(6)
```

Out[7]:
```
'Hello '
```

Iterating Over The Lines of A Text File

You can use a file object as an iterator of lines. A typical example is:

In [8]:
```
for line in open("myfile.txt"):
    print(line, end="")
```

```
Hello world!
How are you?
```

Here, newline character is already included so we said `end=""` to prevent extra newline character added by `print` function.

In [9]:
```
for lineno, line in enumerate(open("myfile.txt")):
    print(lineno, line, end="")
```

```
0 Hello world!
1 How are you?
```

Calling Multiple `.read` / `.write` / `.seek`

If you need to use `.read` / `.write` on the same file multiple times with same mode then instead of openning the file each time needed, just open that once.

In [10]:
```
fd = open("myfile.txt")
```

And call the `.read` / `.write` methods when needed.

In [11]:
```
fd.read(5)
```

Out[11]:
```
'Hello'
```

In [12]:
```
fd.readline()
```

Out[12]:
```
' world!\n'
```

To move to another offset in the file use `.seek` method.

In [13]:
```
fd.seek(0)
fd.readline()
```

Out[13]:
```
'Hello world!\n'
```

after readin/writing all the text, close the file with `.close` method to *flush* the content to free the memory and write the changes to the file system.

In [14]:
```
fd.close()
```

After closing you can not read/write the file.

`with` Statements

Instead of this `open` - `read/write` - `close` sequence, a better way of doing this in Python is to use `with` statement.

In [15]:
```
with open("myfile.html", "w") as fd:
    fd.write("<h1>Simple HTML</h1>")
    fd.write("This is a <b>simple</b> HTML file")
    fd.write("You can open this file using your browser")
```

Here the file is automatically closed when the `width` block is finished. This makes code more readable and prevents you from forgetting to close the file.

Using `print` Function to Write A String to A Text File

Instead of `.write` method, you can also use familiar `print` function to write a string to a file. using `file=` keyword parameter.

In [16]:
```
with open("myfile.html", "w") as fd:
    print("<img src=https://www.python.org/static/img/python-logo.png>", file=fd)
    print("<h1>Simple HTML File</h1>", file=fd)
    print("This is a <b>simple</b> HTML file with an image", file=fd)
    print("<i>You can open this file using your browser</i>", file=fd)
```

Warning: `.write` method is a low level methed of writing a string to a file. Its argument must be a single string without new line character, while the arguments of `print` function can be any number of objects convertable to string. And the newline character (`\n`) is added to the end.

Modes for Opening Files

mode	description
r	open for reading (default)
w	open for writing, truncating the file first
x	open for exclusive creation, failing if the file already exists
a	open for writing, appending to the end of file

Mixed Modes

mode	description
r+	open for reading but can also write
w+	open for writing, truncating the file first but can also read
x+	open for exclusive creation, failing if the file already exists but can also read
a+	open for writing, appending to the end of file but can also read

Opening Files in Binary Mode

To open and read/write files in a lower level use binary modes instead:

mode	description
br	open for reading binary
bw	open for writing binary, truncating the file first
bx	open for exclusive creation binary, failing if the file already exists
ba	open for writing binary, appending to the end of file
br+	open for reading binary but can also write
bw+	open for writing binary, truncating the file first but can also read
bx+	open for exclusive creation binary, failing if the file already exists but can also read
ba+	open for writing binary, appending to the end of file but can also read

Warning: The result of `read` will be a `bytes` object not `str`. `bytes` objects are low level representations of strings.

In [17]:
```
data = open("myfile.txt","rb").read()
data
```

Out[17]:
```
b'Hello world!\nHow are you?'
```

You can do many string-like operations on them.

In [18]:
```
b" ".join((b"Hello",b"world"))
```

Out[18]:
```
b'Hello world'
```

A `bytes` object can be converted to to a `str` object by calling its `.decode()` method.

In [19]:
```
data.decode()
```

Out[19]:
```
'Hello world!\nHow are you?'
```

A `str` object can be converted to a `bytes` object by calling its `.encode()` method.

In [20]:
```
"myfile.txt: ".encode() + data
```

Out[20]:
```
b'myfile.txt: Hello world!\nHow are you?'
```

Warning: The items of a `bytes` object is an integer. Unlike strings. Because the items of a string are also strings.

In [21]:
```
set(data) # data is a bytes object
```

Out[21]:
```
{10, 32, 33, 63, 72, 97, 100, 101, 108, 111, 114, 117, 119, 121}
```

In [22]:
```
set(data.decode())  # data.decode() is a string
```

Out[22]:
```
{'\n', ' ', '!', '?', 'H', 'a', 'd', 'e', 'l', 'o', 'r', 'u', 'w', 'y'}
```